

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

OFFICIAL CURRICULUM & SYLLABUS HANDBOOK (2024-2028)

Branch: Computer Science & Engineering (CSE) | Degree: Bachelor of Technology (B.Tech)

Program Overview & Degree Structure

The Bachelor of Technology in Computer Science and Engineering is a 4-year full-time academic program.

Total Degree Credits: 160 credits across 8 semesters.

Semester 3: Data Structures and Algorithms (CS201, 4 Credits)

Semester 5: Database Management Systems (CS301, 4 Credits), Operating Systems (CS302, 4 Credits), Computer Networks (CS303, 3 Credits)

Semester 6: Compiler Design (CS304, 4 Credits), Web Technologies (CS305, 3 Credits), Software Engineering (CS306, 3 Credits)

Semester 7: Machine Learning (CS401, 4 Credits), Artificial Intelligence (CS402, 3 Credits)

Semester 8: Cloud Computing (CS403, 3 Credits), Capstone Project & Internship (10 Credits)

COURSE SPECIFICATION: DATABASE MANAGEMENT SYSTEMS

Course Code: CS301 | Branch: CSE | Semester: 5 | Credits: 4 (L: 3, T: 1, P: 0)

Prerequisites: Data Structures and Algorithms (CS201), Discrete Mathematics (MA202)

Course Objectives:

- ? To understand the fundamental concepts of relational database management systems.
- ? To design database schemas using Entity-Relationship modeling and normalization techniques.
- ? To formulate complex database queries using Structured Query Language (SQL) and relational algebra.
- ? To examine transaction processing, concurrency control protocols, and database recovery mechanisms.
- ? To introduce NoSQL architectures and modern storage paradigms.

Course Outcomes (COs):

- ? CO1: Apply relational algebra and relational calculus to formulate declarative database queries.
- ? CO2: Design conceptual schema using Entity-Relationship diagrams and map them to relational schemas.
- ? CO3: Construct optimized SQL queries including joins, subqueries, views, and aggregate functions.
- ? CO4: Evaluate relational schemas using 1NF, 2NF, 3NF, and BCNF functional dependency decomposition to eliminate anomalies.
- ? CO5: Analyze ACID transaction properties, serializability, lock-based concurrency control, and write-ahead logging recovery.

Syllabus Breakdown by Units:

Unit 1: Introduction & Database System Architecture

Data models, schemas, and instances; Three-schema architecture and data independence; Database languages and interfaces; Database users and administrators; Entity-Relationship model: entities, attributes, entity sets, relationships, constraints, weak entity sets, Extended ER features (specialization, generalization, categorization).

Unit 2: Relational Data Model & Relational Algebra

Relational model concepts, relational integrity constraints; Relational algebra operations: Selection, Projection, Union, Set Difference, Cartesian Product, Rename, Natural Join, Outer Joins, Division; Tuple Relational Calculus and Domain Relational Calculus; Overview of QBE.

Unit 3: SQL & Query Processing

Data Definition Language (DDL) commands: CREATE, ALTER, DROP; Data Manipulation Language (DML) commands: INSERT, UPDATE, DELETE; Advanced SELECT queries: nested subqueries, correlated subqueries, set operations, aggregate functions, GROUP BY, HAVING; Integrity constraints, assertions, triggers, views; Overview of query optimization and cost estimation.

Unit 4: Database Design Theory & Normalization

Informal design guidelines for relation schemas; Functional dependencies: Armstrong's axioms, closure of functional dependencies, minimal cover; Normal forms based on primary keys: First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF); Boyce-Codd Normal Form (BCNF); Multi-valued dependencies and Fourth Normal Form (4NF); Lossless join decomposition and dependency preservation.

Unit 5: Transaction Processing, Concurrency & Recovery

Transaction concepts, states, ACID properties; Schedules and serializability: conflict and view serializability; Concurrency control techniques: Two-Phase Locking (2PL), strict 2PL, deadlock prevention and detection, timestamp ordering; Database recovery techniques: deferred update, immediate update, shadow paging, write-ahead logging (WAL), checkpoints.

COURSE SPECIFICATION: OPERATING SYSTEMS

Course Code: CS302 | Branch: CSE | Semester: 5 | Credits: 4 (L: 3, T: 1, P: 0)

Prerequisites: Computer Organization and Architecture (CS202), Data Structures and Algorithms (CS201)

Course Objectives:

- ? To introduce the core services, structures, and system call interfaces of modern operating systems.
- ? To explain process management, threads, multi-programming, and CPU scheduling algorithms.
- ? To study process synchronization mechanisms, critical-section problem solutions, and deadlock handling.
- ? To analyze contiguous and non-contiguous memory management, paging, segmentation, and virtual memory.
- ? To examine file system implementation, directory structures, and secondary storage disk scheduling.

Course Outcomes (COs):

- ? CO1: Explain operating system structure, system calls, interrupt handling, and kernel architecture.
- ? CO2: Analyze and compare preemptive and non-preemptive CPU scheduling algorithms for responsiveness and throughput.
- ? CO3: Implement synchronization primitives (semaphores, mutexes, monitors) to solve classic synchronization problems.
- ? CO4: Design and evaluate deadlock prevention, avoidance (Banker's Algorithm), and detection strategies.
- ? CO5: Demonstrate virtual memory techniques including page replacement algorithms (FIFO, LRU, Optimal) and evaluate disk scheduling.

Syllabus Breakdown by Units:

Unit 1: OS Overview & Process Concept

Role and purpose of operating systems, kernel architectures (monolithic, microkernel, hybrid), system calls, OS services; Process concept: process states, Process Control Block (PCB), context switching; Threads: user-level and kernel-level threads, multithreading models; Process scheduling queues, schedulers.

Unit 2: CPU Scheduling & Process Synchronization

Scheduling criteria: CPU utilization, throughput, turnaround time, waiting time, response time; Scheduling algorithms: First-Come First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Priority Scheduling, Round Robin (RR), Multilevel Queue Scheduling; Inter-process communication (IPC); Critical Section problem, Peterson's algorithm, hardware synchronization, semaphores, mutex locks, monitors; Classic problems: Producer-Consumer, Readers-Writers, Dining Philosophers.

Unit 3: Deadlocks

Deadlock characterization: mutual exclusion, hold and wait, no preemption, circular wait; Resource allocation graph (RAG); Deadlock prevention strategies; Deadlock avoidance: safe state, Banker's algorithm for single and multiple resource instances; Deadlock detection and recovery.

Unit 4: Memory Management & Virtual Memory

Logical versus physical address space, swapping, contiguous allocation, fragmentation (internal and external), paging, translation lookaside buffer (TLB), multi-level paging, inverted page tables, segmentation; Virtual memory: demand paging, page fault handling, page replacement algorithms (FIFO, Optimal, LRU, LFU, MFU), allocation of frames, thrashing and working set model.

Unit 5: Storage Management & File Systems

File concepts, access methods (sequential, direct, indexed), directory structures, file mounting and protection; File system structure, allocation methods (contiguous, linked, indexed), free-space management; Secondary storage structure: disk structure, disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK), RAID levels.

COURSE SPECIFICATION: COMPUTER NETWORKS

Course Code: CS303 | Branch: CSE | Semester: 5 | Credits: 3 (L: 3, T: 0, P: 0)

Prerequisites: Data Communication Principles (CS204), Operating Systems (CS302)

Course Objectives:

- ? To build a solid understanding of layered network architectures (OSI 7-layer and TCP/IP protocol suites).
- ? To study framing, error detection/correction, and medium access control mechanisms.
- ? To master routing algorithms, subnetting, IPv4/IPv6 addressing, and congestion control.
- ? To explore transport protocols (TCP and UDP), flow control, reliability, and socket programming.
- ? To analyze key application layer protocols including DNS, HTTP, SMTP, and network security basics.

Course Outcomes (COs):

- ? CO1: Differentiate OSI reference model and TCP/IP protocol stack layers and their respective duties.
- ? CO2: Apply error detection (CRC, checksum) and framing algorithms at the Data Link Layer.
- ? CO3: Formulate IPv4/IPv6 CIDR subnet masks and evaluate distance-vector and link-state routing protocols.
- ? CO4: Contrast TCP and UDP connection semantics, congestion control (AIMD, slow start), and flow control.
- ? CO5: Analyze client-server application protocols including DNS, HTTP/HTTPS, DHCP, and SMTP.

Syllabus Breakdown by Units:

Unit 1: Introduction & Physical / Data Link Layer

Network topologies, PAN, LAN, MAN, WAN; Protocol layers: OSI reference model vs TCP/IP protocol suite; Physical layer transmission media; Data link layer design issues: framing, error control, CRC, Hamming codes, flow control: Stop-and-Wait, Go-Back-N, Selective Repeat ARQ; HDLC.

Unit 2: Medium Access Control (MAC) Sublayer

Multiple access protocols: Pure and Slotted ALOHA, CSMA, CSMA/CD, CSMA/CA; IEEE 802.3 Ethernet standard, Fast Ethernet, Gigabit Ethernet; IEEE 802.11 Wireless LAN architecture; Bridges, switches, VLANs.

Unit 3: Network Layer & Routing

Network layer design issues, store-and-forward packet switching; Routing algorithms: shortest path routing, Distance Vector Routing (Bellman-Ford), Link State Routing (Dijkstra), hierarchical routing, OSPF, BGP; Congestion control principles; IP addressing: IPv4 classes, CIDR subnetting, NAT, ARP, RARP, ICMP, IPv6 header format and transition strategies.

Unit 4: Transport Layer

Transport service primitives, socket addressing, multiplexing and demultiplexing; User Datagram Protocol (UDP): segment header, checksum; Transmission Control Protocol (TCP): connection establishment (three-way handshake), termination, TCP segment structure, flow control via sliding window, TCP congestion control: slow start, congestion avoidance, fast retransmit, fast recovery.

Unit 5: Application Layer & Security Fundamentals

Domain Name System (DNS): hierarchical namespace, resource records, resolution; Electronic mail: SMTP, POP3, IMAP; HyperText Transfer Protocol (HTTP/1.1, HTTP/2), HTTPS, cookies; Dynamic Host Configuration Protocol (DHCP); Network security introduction: symmetric and asymmetric cryptography, firewalls.

COURSE SPECIFICATION: COMPILER DESIGN

Course Code: CS304 | Branch: CSE | Semester: 6 | Credits: 4 (L: 3, T: 1, P: 0)

Prerequisites: Formal Languages and Automata Theory (CS205), Data Structures and Algorithms (CS201)

Course Objectives:

- ? To explore the phases of a language translator and the construction of lexical analyzers.
- ? To develop top-down and bottom-up parsing techniques for Context-Free Grammars.
- ? To apply syntax-directed translation for intermediate code generation.
- ? To study code optimization algorithms and target machine code generation.

Course Outcomes (COs):

- ? CO1: Construct deterministic finite automata and lexical analyzers using regular expressions.
- ? CO2: Design LL(1), SLR(1), LR(1), and LALR(1) parsing tables for context-free grammars.
- ? CO3: Formulate syntax-directed definitions (SDD) and generate intermediate representations like three-address code.
- ? CO4: Apply machine-independent optimizations including loop invariant code motion and common subexpression elimination.
- ? CO5: Generate optimized target machine instructions and implement register allocation using graph coloring.

Syllabus Breakdown by Units:

Unit 1: Introduction to Compilers & Lexical Analysis

Phases of a compiler, passes, compiler construction tools; Role of lexical analyzer, tokens, patterns, lexemes; Regular expressions, Thompson's construction to NFA, subset construction to DFA, minimization of DFAs; Lex/Flex tool specification.

Unit 2: Syntax Analysis (Top-Down & Bottom-Up Parsing)

Context-free grammars, derivations, parse trees, ambiguity; Top-down parsing: recursive descent, predictive LL(1) parsing, FIRST and FOLLOW computations; Bottom-up parsing: shift-reduce parsing, operator precedence, LR parsing algorithms (SLR, canonical LR, LALR); Yacc/Bison parser generator.

Unit 3: Syntax-Directed Translation & Intermediate Code

Syntax-directed definitions (SDD), synthesized and inherited attributes, evaluation orders for SDDs; Intermediate representations: syntax trees, postfix notation, three-address code (quadruples, triples, indirect triples); Translation of expressions, type checking, boolean expressions, control flow statements.

Unit 4: Run-Time Storage & Code Optimization

Storage organization, activation records, stack allocation of space, heap management, parameter passing mechanisms; Principles of code optimization: basic blocks, flow graphs, loop optimization, common subexpression elimination, dead code elimination, copy propagation, constant folding.

Unit 5: Code Generation & Register Allocation

Issues in the design of code generators; Target machine description; Simple code generator algorithm; Register allocation and assignment, register allocation by graph coloring; Peephole optimization.

COURSE SPECIFICATION: MACHINE LEARNING

Course Code: CS401 | Branch: CSE | Semester: 7 | Credits: 4 (L: 3, T: 1, P: 0)

Prerequisites: Probability and Linear Algebra (MA201), Data Structures and Algorithms (CS201), Python for Data Science (CS102)

Course Objectives:

- ? To introduce fundamental concepts of supervised, unsupervised, and reinforcement learning.
- ? To study mathematical foundations of regression, classification, and clustering algorithms.
- ? To evaluate model generalization, bias-variance tradeoff, and cross-validation methodologies.
- ? To introduce neural networks, deep learning architectures, and modern ML pipelines.

Course Outcomes (COs):

- ? CO1: Formulate supervised learning models including Linear Regression, Logistic Regression, and SVMs.
- ? CO2: Analyze decision trees, random forests, and ensemble boosting techniques (AdaBoost, XGBoost).
- ? CO3: Apply unsupervised clustering (K-Means, Hierarchical) and dimensionality reduction (PCA, t-SNE).
- ? CO4: Evaluate classification performance using precision, recall, F1 score, ROC-AUC, and cross-validation.
- ? CO5: Construct multi-layer perceptrons with backpropagation and explain regularizers (L1/L2, dropout).

Syllabus Breakdown by Units:

Unit 1: Foundations & Supervised Regression

Overview of machine learning, types of learning; Linear regression: hypothesis representation, ordinary least squares, gradient descent (batch, stochastic, mini-batch); Polynomial regression; Overfitting, underfitting, bias-variance tradeoff; Regularization: Ridge (L2) and Lasso (L1) regression.

Unit 2: Supervised Classification Models

Logistic regression: sigmoid activation, log-loss cost function, decision boundaries; Naive Bayes classifier, Bayes theorem, Gaussian and Multinomial variants; Support Vector Machines (SVM): maximal margin hyperplane, soft margin, kernel trick (RBF, polynomial kernels); K-Nearest Neighbors (KNN).

Unit 3: Tree-Based Models & Ensemble Learning

Decision tree learning: ID3, C4.5, CART; Splitting criteria: entropy, information gain, Gini impurity; Tree pruning; Ensemble methods: Bagging (Random Forest), Boosting (AdaBoost, Gradient Boost, XGBoost); Stacking and voting classifiers.

Unit 4: Unsupervised Learning & Dimensionality Reduction

Clustering: K-Means algorithm, Elbow method, Silhouette score; Hierarchical clustering (agglomerative, divisive, dendrograms); Density-based clustering (DBSCAN); Dimensionality reduction: Principal Component Analysis (PCA), singular value decomposition (SVD), t-SNE.

Unit 5: Neural Networks & Model Evaluation

Biological vs artificial neurons, Perceptron learning rule; Multi-Layer Perceptron (MLP), forward propagation, activation functions (ReLU, Sigmoid, Tanh, Softmax); Backpropagation algorithm, chain rule, loss functions; Evaluation metrics: confusion matrix, precision, recall, F1-score, ROC and PR curves, k-fold cross-validation.

COURSE SPECIFICATION: DATA STRUCTURES AND ALGORITHMS

Course Code: CS201 | Branch: CSE | Semester: 3 | Credits: 4 (L: 3, T: 1, P: 0)

Prerequisites: Programming in C (CS101)

Course Objectives:

- ? To introduce asymptotic notations and complexity analysis of algorithms.
- ? To implement linear data structures: arrays, linked lists, stacks, and queues.
- ? To master non-linear data structures: trees, binary search trees, AVL trees, and graphs.
- ? To apply classic algorithm design paradigms: divide-and-conquer, greedy, and dynamic programming.

Course Outcomes (COs):

- ? CO1: Compute asymptotic time and space complexity using Big-O, Big-Omega, and Big-Theta notations.
- ? CO2: Implement singly, doubly, and circular linked lists, stacks, and queues for real-world problems.
- ? CO3: Construct binary trees, BSTs, and self-balancing AVL trees with tree traversals.
- ? CO4: Execute graph traversal algorithms (BFS, DFS) and minimum spanning tree algorithms (Prim, Kruskal).
- ? CO5: Design dynamic programming and greedy solutions (knapsack, Dijkstra, Floyd-Warshall).

Syllabus Breakdown by Units:

Unit 1: Complexity Analysis & Linear Data Structures

Asymptotic notations: O, Omega, Theta; Arrays and matrices; Linked Lists: singly linked lists, doubly linked lists, circular linked lists; Operations: insertion, deletion, searching, traversal.

Unit 2: Stacks & Queues

Stack abstract data type, array and linked list implementation; Applications of stacks: infix to postfix conversion, postfix evaluation, balanced parenthesis; Queue ADT: circular queues, double-ended queues (deque), priority queues.

Unit 3: Trees & Binary Search Trees

Tree terminologies, binary trees, representations, binary tree traversals (inorder, preorder, postorder, level order); Binary Search Trees (BST): insertion, deletion, searching; Balanced search trees: AVL trees, rotations; B-trees and heaps (min-heap, max-heap).

Unit 4: Graphs & Graph Algorithms

Graph representations: adjacency matrix, adjacency list; Graph traversals: Breadth First Search (BFS), Depth First Search (DFS); Topological sort; Minimum Spanning Trees: Prim's and Kruskal's algorithms; Shortest paths: Dijkstra's algorithm.

Unit 5: Algorithm Design Techniques & Sorting

Divide and Conquer: Merge Sort, Quick Sort; Greedy algorithms: Fractional Knapsack, Huffman coding; Dynamic Programming: 0/1 Knapsack, Longest Common Subsequence; Sorting algorithms: Insertion, Selection, Bubble, Radix sort; Hashing: hash tables, collision resolution techniques.

COURSE SPECIFICATION: WEB TECHNOLOGIES

Course Code: CS305 | Branch: CSE | Semester: 6 | Credits: 3 (L: 3, T: 0, P: 0)

Prerequisites: Object-Oriented Programming through Java (CS203)

Course Objectives:

- ? To teach modern web architectural standards, semantic markup, and responsive stylesheets.
- ? To explore client-side scripting with JavaScript and DOM manipulation.
- ? To build RESTful web services and server-side applications.

Course Outcomes (COs):

- ? CO1: Construct standards-compliant, accessible responsive layouts using HTML5 and CSS3.
- ? CO2: Write modern ES6+ JavaScript code handling events, asynchronous fetch calls, and DOM modifications.
- ? CO3: Develop backend server APIs using Node.js and Express with JSON payload exchanges.
- ? CO4: Integrate relational and document databases with backend web application routes.
- ? CO5: Implement web security best practices including CORS, CSRF, XSS protection, and JWT authentication.

Syllabus Breakdown by Units:

Unit 1: HTML5, CSS3 & Responsive Design

HTML5 semantic elements, forms, input types, audio and video tags; CSS3 selectors, box model, Flexbox, CSS Grid layout, media queries, mobile-first design principles.

Unit 2: Client-Side Scripting with JavaScript (ES6+)

JavaScript syntax, variables (let, const), data types, functions, arrow functions, template literals, destructuring; DOM manipulation, event listeners; Promises, async/await, Fetch API for AJAX communication.

Unit 3: Server-Side Programming with Node.js & Express

Node.js runtime environment, npm, event loop, modules; Express framework: routing, middleware, request and response objects, error handling, serving static files.

Unit 4: RESTful API Architecture & Data Persistence

REST architectural constraints, HTTP methods (GET, POST, PUT, DELETE), status codes, JSON serialization; Database connectivity with MongoDB and MySQL; CRUD operations via ORMs.

Unit 5: Web Security & Deployment

Web security vulnerabilities: Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL injection; Authentication & Authorization using JWT and OAuth2; Deployment on cloud platforms, HTTPS, SSL/TLS certificates.

COURSE SPECIFICATION: SOFTWARE ENGINEERING

Course Code: CS306 | Branch: CSE | Semester: 6 | Credits: 3 (L: 3, T: 0, P: 0)

Prerequisites: None

Course Objectives:

- ? To introduce software development lifecycles and Agile engineering practices.
- ? To master requirements elicitation, SRS authoring, and architectural design modeling.
- ? To apply software testing methodologies and quality assurance standards.

Course Outcomes (COs):

- ? CO1: Select appropriate software process models (Waterfall, Spiral, Agile Scrum) for software projects.
- ? CO2: Draft formal Software Requirements Specifications (SRS) based on IEEE 830 guidelines.
- ? CO3: Design modular software architectures using UML class, use-case, sequence, and activity diagrams.
- ? CO4: Formulate black-box and white-box test cases including boundary value analysis and path coverage.
- ? CO5: Apply software maintenance, risk management, and project scheduling techniques (Gantt charts, PERT).

Syllabus Breakdown by Units:

Unit 1: Software Process Models

Nature of software, software engineering layers; Software Process Models: Waterfall, Incremental, Evolutionary, Spiral; Agile development: Agile manifesto, Scrum methodology, sprint planning, user stories, extreme programming (XP).

Unit 2: Requirements Engineering & Analysis

Requirements engineering tasks: inception, elicitation, elaboration, negotiation, specification, validation; Functional and non-functional requirements; Use-case modeling; IEEE 830 standard Software Requirements Specification (SRS) documentation.

Unit 3: Software Design & UML Modeling

Design concepts: abstraction, modularity, architecture, information hiding, functional independence (cohesion and coupling); Object-Oriented design using UML: Class diagrams, Object diagrams, Sequence diagrams, Collaboration diagrams, Activity diagrams, State machine diagrams.

Unit 4: Software Testing Strategies & Techniques

Strategic approach to testing: unit testing, integration testing, validation testing, system testing; Black-box testing: equivalence partitioning, boundary value analysis, cause-effect graphing; White-box testing: basis path testing, cyclomatic complexity, control structure testing; Regression testing.

Unit 5: Software Project Management & Quality

Project metrics, software estimation: COCOMO II model; Project scheduling: Gantt charts, PERT/CPM; Risk management: identification, projection, mitigation; Software Quality Assurance (SQA): ISO 9001, CMMI levels, software maintenance.

COURSE SPECIFICATION: ARTIFICIAL INTELLIGENCE

Course Code: CS402 | Branch: CSE | Semester: 7 | Credits: 3 (L: 3, T: 0, P: 0)

Prerequisites: Data Structures and Algorithms (CS201), Discrete Mathematics (MA202)

Course Objectives:

- ? To provide foundations of intelligent agents and heuristic search strategies.
- ? To model knowledge representation and automated reasoning using first-order predicate logic.
- ? To explore planning algorithms and uncertainty management using Bayesian networks.

Course Outcomes (COs):

- ? CO1: Formulate state-space search problems for rational agents in complex environments.
- ? CO2: Implement informed heuristic search algorithms including A* search and Greedy Best-First Search.
- ? CO3: Represent domain knowledge using First-Order Logic (FOL) and apply resolution refutation.
- ? CO4: Model probabilistic reasoning in uncertain domains using Bayesian Belief Networks.
- ? CO5: Design adversarial game-playing agents using Minimax algorithm with Alpha-Beta pruning.

Syllabus Breakdown by Units:

Unit 1: Introduction to AI & State-Space Search

Definitions of AI, Turing test, intelligent agents, agent architectures, PEAS classification, environment types; Problem formulation as state-space search; Uninformed search strategies: Breadth-First Search (BFS), Depth-First Search (DFS), Uniform-Cost Search (UCS), Iterative Deepening Search (IDS).

Unit 2: Heuristic Search & Adversarial Games

Informed search strategies: Greedy Best-First Search, A* search, admissibility and consistency of heuristics; Local search: Hill climbing, Simulated Annealing, Genetic Algorithms; Adversarial search: Game playing, Minimax algorithm, Alpha-Beta pruning, evaluation functions.

Unit 3: Knowledge Representation & Logic

Knowledge-based agents, propositional logic, inference rules, horn clauses; First-Order Logic (FOL): syntax and semantics, quantifiers, unifications, forward chaining, backward chaining, resolution refutation theorem proving.

Unit 4: Classical Planning & Constraint Satisfaction

Constraint Satisfaction Problems (CSP): constraint propagation, backtracking search for CSPs, forward checking; Classical planning: PDDL language, STRIPS operators, state-space planning, planning graphs.

Unit 5: Reasoning under Uncertainty

Acting under uncertainty, probability axioms, joint probability distribution, conditional probability, Bayes' rule; Bayesian Belief Networks: semantics, conditional independence; Exact and approximate inference in Bayesian networks.

COURSE SPECIFICATION: CLOUD COMPUTING

Course Code: CS403 | Branch: CSE | Semester: 8 | Credits: 3 (L: 3, T: 0, P: 0)

Prerequisites: Operating Systems (CS302), Computer Networks (CS303)

Course Objectives:

- ? To study cloud delivery models (IaaS, PaaS, SaaS) and deployment architectures.
- ? To examine hypervisor virtualization and container technologies (Docker, Kubernetes).
- ? To analyze cloud storage, distributed file systems, serverless computing, and cloud security.

Course Outcomes (COs):

- ? CO1: Differentiate cloud service models (IaaS, PaaS, SaaS) and deployment types (Public, Private, Hybrid).
- ? CO2: Analyze CPU, memory, and network virtualization techniques and hypervisors (Type 1 and Type 2).
- ? CO3: Deploy containerized microservices using Docker and manage container clusters with Kubernetes.
- ? CO4: Examine distributed storage architectures including Google File System (GFS) and Amazon S3.
- ? CO5: Implement cloud security, IAM role-based access control, and SLA monitoring.

Syllabus Breakdown by Units:

Unit 1: Cloud Computing Fundamentals

NIST definition of cloud computing, essential characteristics, service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), Software as a Service (SaaS); Deployment models: Public, Private, Community, Hybrid; Economics of cloud.

Unit 2: Virtualization Technology

Virtualization concepts, benefits, hardware-assisted virtualization; Hypervisors: Type-1 (bare-metal) and Type-2 (hosted); Virtualization of CPU, memory, and I/O devices; Virtual machine migration (live migration); Comparison between VMs and Containers.

Unit 3: Containers & Cloud Orchestration

Linux namespaces and cgroups; Docker container architecture, Dockerfile, Docker Compose; Container orchestration using Kubernetes: pods, services, deployments, ingress controllers, horizontal pod autoscaling (HPA).

Unit 4: Cloud Storage & Distributed Computing

Distributed file systems: Google File System (GFS), Hadoop Distributed File System (HDFS); Cloud object storage (Amazon S3); NoSQL in the cloud; Serverless computing and Function-as-a-Service (AWS Lambda, Google Cloud Functions).

Unit 5: Cloud Security, Governance & SLAs

Cloud security risks, shared responsibility model, Identity and Access Management (IAM); Encryption at rest and in transit; Service Level Agreements (SLAs), cloud monitoring and auto-scaling, disaster recovery in the cloud.